

StockFeelings: A RAG-Based Sentiment Analysis System for Stock Trading

Arvind Sudhakar Badgujar, Courage Ochuko Mene, Jubal Chandy Jacob, Paul Osemudiamé Oamen, Zhanel Garifullina
 {t29ab24, c.mene25, t13jc24, p.oamen25, t01zg25}@abdn.ac.uk
 University of Aberdeen

December 2025

N = 1470 (word count excluding figures, captions, tables, and listings).

Abstract

StockFeelings is an AI-powered sentiment analysis platform that combines real-time sentiment extraction from multiple sources with Retrieval-Augmented Generation (RAG) for historical pattern matching. Unlike existing tools that focus solely on sentiment scoring, StockFeelings bridges sentiment analysis with historical context by using cosine similarity to match current market conditions against past patterns, providing interpretable insights grounded in historical market behaviour.

1. Introduction and Project Definition

In October 2025, Alphabet lost \$150 billion in market value within a single trading day following OpenAI’s announcement of the ChatGPT Atlas browser (Sigalos, 2025). Nothing about Google’s products, services, or underlying fundamentals had changed, yet *investor sentiment* about a potential competitive threat was enough to wipe out billions. This illustrates a fundamental market reality where perception can move markets as powerfully as performance, a principle long affirmed by successful investors such as Warren Buffett through his famous maxim on fear and greed (Buffett, 2008). As such, sentiment analysis tools have become as essential as traditional trading indicators for investors at every scale.

This paper presents **StockFeelings**, an AI-powered sentiment analysis system designed to help stock investors and traders make informed trading decisions using real-time sentiment and historical context.

1.1 Our Innovation

While numerous AI-powered sentiment analysis tools exist today, most treat sentiment scoring and price prediction as isolated tasks (Bollen et al., 2011; Pagolu et al., 2016). **StockFeelings** introduces a novel approach by integrating Retrieval-Augmented Generation (RAG) with multi-source sentiment analysis (P. Lewis et al., 2020). The system encodes current market sentiment alongside common thematic patterns into embedding vectors and

employs cosine similarity to retrieve historically similar conditions, effectively answering the question: “*When sentiment looked like this before, what happened next?*” This historical pattern matching, combined with intelligent source-weighted aggregation, forms the core contribution of this work.

Table 1 and Table 2 show the comparison with existing platforms and core formulations respectively. Figure 1 shows the system architecture.

Table 1: Comparison with some existing platforms

Value	StockFeelings	Sentifi	StockTwits	MarketPsych	CNN F&G
Real-time sentiment	✓	✓	✓	✓	✓
Multi-source	✓	✓	–	✓	–
AI/NLP-based	✓	✓	–	✓	–
Stock-level coverage	✓	✓	✓	✓	–
Theme extraction	✓	–	–	✓	–
Historical context	✓	–	–	–	–
Price correlation	✓	–	–	–	–
Retail accessible	✓	–	✓	–	✓

Table 2: Core algorithmic formulations. α and β are source weights ($\alpha + \beta = 1$), $\mathbf{q}$ is the query vector, $\mathbf{d}_i$ are historical vectors in FAISS, and ΔP_j is price movement after period j .

Component	Formula
Source Tagging	$x_i = [\text{TAG}] \oplus \text{text}_i$
Sentiment Score	$s_i = f_{\text{DistilRoBERTa}}(x_i) \in [-1, 1]$
Weighted Aggregation	$S_{\text{agg}} = \alpha \bar{S}_{\text{news}} + \beta \bar{S}_{\text{social}}$, $\alpha > \beta$
Cosine Similarity	$\text{sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\ \mathbf{q}\ \ \mathbf{d}_i\ }$
Top-k Retrieval	$\mathcal{D}_k = \text{top-}k \text{ sim}(\mathbf{q}, \mathbf{d}_i)_{\mathbf{d}_i \in \mathcal{D}}$
Historical ΔP	$\Delta P_{\text{hist}} = \frac{\sum_{j=1}^k \text{sim}_j \cdot \Delta P_j}{\sum_{j=1}^k \text{sim}_j}$
Classification	$\text{Label} = \begin{cases} \text{Bullish} & S_{\text{agg}} > \tau \\ \text{Bearish} & S_{\text{agg}} < -\tau \\ \text{Neutral} & \text{otherwise} \end{cases}$

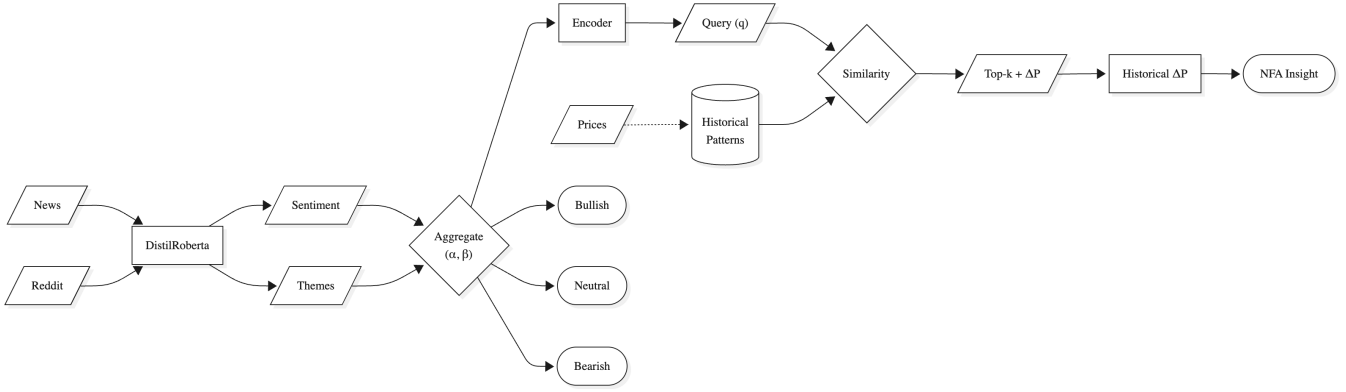


Figure 1: StockFeelings RAG pipeline architecture. Data flows from collection through sentiment processing, RAG-based historical matching, and finally output generation.

2. Experimental setup and configuration

Table 3: Experimental setup and configuration

Category	Specification
<i>Compute Environment</i>	
Platform	Google Colab Enterprise
Machine Type	e2-standard-4 (4 vCPU, 16GB)
Region	us-central1
Python Version	3.10.11
<i>Data Sources</i>	
News APIs	Google News RSS, Yahoo Finance, Finnhub
Social Media	Reddit JSON API (23 sub-reddits)
Stock Prices	Polygon.io (5-min bars)
Collection Period	Nov 2024 – Nov 2025
<i>Dataset Statistics</i>	
News Articles	5,118
Reddit Posts	6,521
Total Samples	11,639
Price Data Points	54,642 (5-min bars)
Target Tickers	NVDA, TSLA, AAPL, AMD, PLTR
<i>Model Configuration</i>	
Base Model	DistilRoBERTa-base
Max Sequence Length	256 tokens
Batch Size	16
Learning Rate	2×10^{-5}
Training Epochs	4
Warmup Steps	500
Weight Decay	0.01
Optimiser	AdamW
<i>Dataset Split</i>	
Total Samples	11,179 (after preprocessing)
Training Set	7,825 samples (70%)
Validation Set	1,677 samples (15%)
Test Set	1,677 samples (15%)
<i>Key Libraries</i>	
Deep Learning	PyTorch, Transformers
Data Processing	pandas, NumPy, scikit-learn
Data Collection	feedparser, requests, yfinance
Visualisation	matplotlib, seaborn

3. Data

StockFeelings fine-tunes a DistilRoBERTa model on financial news articles and Reddit posts covering five major technology stocks over a 12-month period.

Table 4: Dataset summary

Source	Method	Records	Period
Google News	RSS/feedparser	3,429	Dec 2024 – Nov 2025
Yahoo Finance	yfinance API	452	Nov 2024 – Nov 2025
Finnhub	REST API	1,237	Nov 2024 – Nov 2025
Reddit	JSON API	6,521	Jun 2025 – Nov 2025
Total Text	–	11,639	–

Target Tickers: NVDA, TSLA, AAPL, AMD, PLTR

3.1 Data Collection

3.1.1 News Scraping

News articles were collected from Google News RSS feeds using the `feedparser` library. For each ticker, eight query variations (e.g., “{ticker} stock”, “{ticker} earnings”) maximise coverage. The scraper extracts publication datetime, title, summary, and source from each entry. Rate limiting of 1 second between requests prevents throttling, while deduplication removes identical titles across tickers. Additional data from Yahoo Finance and Finnhub APIs supplement the collection.

```

1 def build_google_news_url(query: str) -> str:
2     formatted_query = query.replace(" ", "+")
3     url = (
4         f"https://news.google.com/rss/search?"
5         f"q={formatted_query}&hl=en-US&gl=US&"
6         f"ceid=US:en"
7     )
8     return url

```

Listing 1: Google News RSS URL construction

```

1 def fetch_rss_feed(url: str) -> Optional[
2     feedparser.FeedParserDict]:
3     try:
4         feed = feedparser.parse(url)
5         if feed.bozo and feed.bozo_exception:
6             return None
7         return feed
8     except Exception as e:
9         print(f"[ERROR] Failed to fetch feed: {
10             e}")
11         return None

```

Listing 2: RSS feed fetching with error handling

3.1.2 Social Media Scraping

Reddit posts were collected via HTTP requests to Reddit’s JSON API across 22 subreddits. Company-specific subreddits (e.g., r/teslainvestorsclub, r/nvidia) capture focused discussions, while general subreddits (r/wallstreetbets, r/stocks) provide broader sentiment. A keyword filter matches company-relevant terms, and quality filters exclude meme phrases (“diamond hands”, “to the moon”) and short titles.

```

1 company_subs = {
2     'TSLA': ['teslainvestorsclub', 'teslamotors',
3             'elonmusk'],
4     'AAPL': ['apple', 'applestocks', 'mac', 'iphone'],
5     'PLTR': ['PLTR', 'Palantir_Investors'],
6     'AMD': ['AMD_Stock', 'Amd', 'AyyMD'],
7     'NVDA': ['nvidiastock', 'nvidia', 'allthingsnvidia']
8 }

```

Listing 3: Subreddit configuration for targeted collection

```

1 def get_subreddit_posts(subreddit, sort='new',
2     after=None):
3     base_url = f"https://www.reddit.com/r/{subreddit}/{sort}.json"
4     params = {'limit': 100, 't': 'year'}
5     if after:
6         params['after'] = after
7     headers = {'User-Agent': 'Mozilla/5.0'}
8     response = requests.get(base_url, params=params,
9                             headers=headers)
10    if response.status_code == 429:
11        time.sleep(60) # Rate limited - wait and retry
12    return get_subreddit_posts(subreddit, sort, after)
13    if response.status_code == 200:
14        return response.json()
15    return None

```

Listing 4: Reddit JSON API request with rate limit handling

3.1.3 Data Merging

News and Reddit data were merged into a unified schema with columns: ticker, datetime, title, label, and

source. The source column distinguishes “news” from “reddit” origins for source-weighted aggregation.

```

1 # Merge news and reddit dataframes
2 df_news['source'] = 'news'
3 df_reddit['source'] = 'reddit'
4
5 merged_df = pd.concat([df_news, df_reddit],
6                       ignore_index=True)
7 merged_df = merged_df.drop_duplicates(
8     subset=['ticker', 'datetime', 'title'],
9     keep='first')
10
11 merged_df = merged_df.sort_values('datetime',
12                                   ascending=False)

```

Listing 5: Unified data merging pipeline

3.2 Preprocessing

We apply a uniform preprocessing pipeline across news and Reddit data, harmonising schema, removing noise, and normalising text before modelling. Table 5 summarises all cleaning operations, implementations, and motivations for the final training corpus.

3.3 Data Labelling

Dataset labels were produced by five student annotators under a controlled-overlap protocol with adjudication of disagreements, yielding high-quality consensus sentiment labels for training and evaluation.

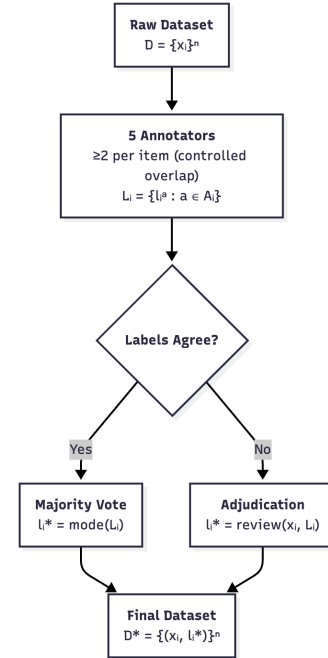


Figure 2: Human annotation workflow. Five annotators label each item with ≥ 2 overlap per item. Items with agreement resolve via majority vote; disagreements undergo adjudication.

Table 5: Text preprocessing operations applied to news and Reddit data.

Scope	Operation	Implementation	Justification
News/Reddit	Deduplication	<code>drop_duplicates(subset=[ticker, datetime, title])</code>	Prevents duplicate articles from biasing label distribution and inflating sample size due to overlapping API queries or mirrored sources.
News/Reddit	Missing-value removal	<code>dropna(subset=[title, ticker, datetime])</code> and <code>title.str.strip() != ""</code>	Ensures every instance has valid identifier, timestamp, and non-empty text for reliable embeddings and temporal joins.
News/Reddit	Datetime parsing & sorting	<code>pd.to_datetime(..., errors='coerce')</code> then sort by <code>datetime</code>	Normalises timestamp formats and enforces chronological order for aligning sentiment with 5-minute price bars.
News/Reddit	HTML decoding	<code>html.unescape(title)</code>	Converts encoded entities (e.g., <code>&amp;</code>) into natural text for proper tokenization without artificial splits.
News/Reddit	Emoji removal	Unicode regex in <code>remove_emojis()</code>	Strips high-variance emoji characters sparsely represented in pre-trained vocabulary, reducing OOV noise.
News/Reddit	URL removal	<code>re.sub(r'http[s]?://\S+', "", title)</code>	Removes non-informative links that waste context window budget and harm embedding quality.
News/Reddit	Cashtag removal	<code>re.sub(r'\$[A-Z]{1,5}\b', "", title)</code>	Eliminates ticker cashtags already in the <code>ticker</code> field, avoiding label leakage.
News	Source-name removal	Regex over common publishers (Reuters, Bloomberg, CNBC, etc.)	Removes boilerplate signatures that bias embeddings towards source style rather than content.
News/Reddit	Whitespace normalisation	<code>re.sub(r'\s+', ' ', text).strip()</code>	Collapses spurious spacing for stable tokenisation and robust string-based filtering.
News	Minimum length filter	Keep titles with <code>len(title) > 10</code>	Excludes fragmentary headlines lacking sufficient lexical context for reliable sentiment inference.
Reddit	Title quality filter	<code>is_high_quality_title()</code> (min word count, no meme phrases)	Filters low-information posts dominated by memes, improving semantic richness.
Reddit	Company keyword filter	<code>check_keywords_match(combined_text, keywords[ticker])</code>	Ensures posts genuinely discuss the target company, reducing off-topic noise.
Reddit	Title-body merging	<code>combined_text = title_clean + ' ' + selftext_clean</code>	Concatenates titles and bodies to maximise context while keeping a single <code>text</code> feature.
News/Reddit	Source tagging	Ensure <code>source</code> column; fill missing as 'news' or subreddit name	Marks document origin for source-aware aggregation and diagnostic analysis.

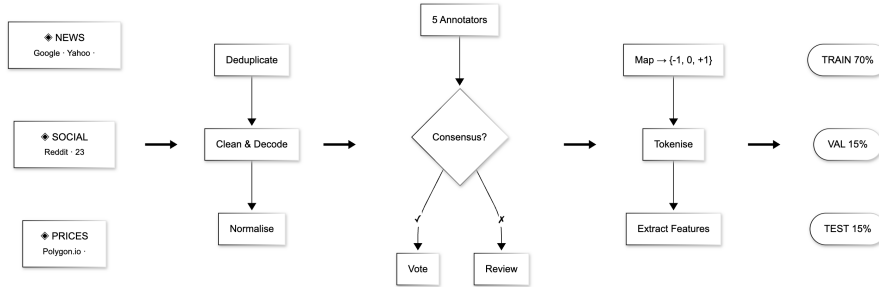


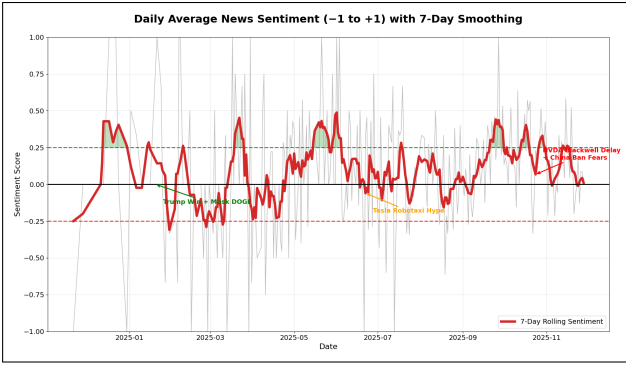
Figure 3: End-to-end data collection and processing pipeline from raw sources to model-ready dataset splits.

Table 6: Label engineering pipeline from raw annotations to model-ready classes.

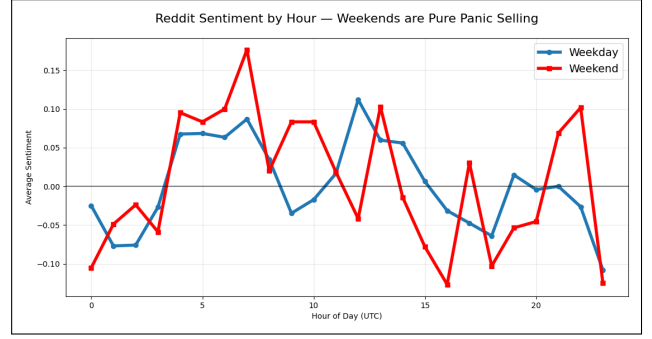
Step	Implementation	Justification
Sentiment string normalisation	Lowercase and strip: <code>df['sentiment'] = df['sentiment'].str.lower().str.strip()</code>	Collapses formatting variation (e.g., "Positive", " POS ") so identical labels map to the same token.
Mapping heterogeneous labels to $\{-1, 0, 1\}$	Rule-based <code>map_to_numeric()</code> : {"positive", "pos", "bullish"} $\rightarrow$ 1, {"neutral", "neu"} $\rightarrow$ 0, {"negative", "neg", "bearish"} $\rightarrow$ -1	Consolidates diverse annotation formats into a single three-class sentiment space for consistent supervision.
Dropping unmappable labels	<code>df = df.dropna(subset=['label'])</code> after mapping	Removes ambiguous labels (e.g., "mixed", sarcasm markers), trading small data loss for clearer semantics.
Casting to integer	<code>df['label'] = df['label'].astype(int)</code>	Enforces numerical type compatible with PyTorch loss functions and stratified splitting.
Model label mapping	<code>Config.LABEL_MAP = {'negative': -1, 'neutral': 0, 'positive': 1}</code> and <code>ID_TO_LABEL</code>	Keeps model-internal IDs aligned with preprocessing scheme for simplified metrics and interpretability.
Dropping post-mapping errors	Remove rows where <code>sentiment</code> not in <code>Config.LABEL_MAP</code>	Ensures training set contains only interpretable labels, preventing silent failures.

Table 7: Feature engineering for text inputs and auxiliary market signals.

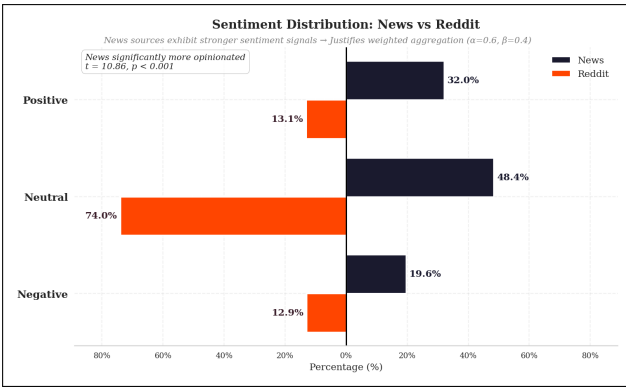
Component	Feature / Implementation	Justification
Text schema standardisation	Auto-detect text column via heuristics (<code>text</code> , <code>title</code> , <code>headline</code>) and rename to <code>text</code>	Enables single pipeline to ingest differently named columns without manual reconfiguration.
Text field construction	Use cleaned news <code>title</code> or Reddit <code>combined_text</code> as sole <code>text</code> feature	Provides concise yet information-rich representation tailored to each source type.
Tokenisation	DistilRoBERTa tokenizer: <code>max_length=256</code> , <code>padding='max_length'</code> , <code>truncation=True</code>	Converts text into transformer-ready representations with capped sequence length for GPU memory.
Removal of raw text	<code>dataset.remove_columns(['text'])</code> after tokenisation	Prevents leakage of raw text into model, reduces memory footprint.
Stratified dataset split	<code>train_test_split(..., stratify=df['label'])</code> into 70/15/15	Maintains class balance across splits for genuine performance evaluation.
Price returns	Simple: <code>close.pct_change()</code> ; Log: <code>log(close / close.shift(1))</code>	Quantifies short-horizon price movements for analysing sentiment-price relationships.
Rolling volatility	1-hour volatility: <code>returns.rolling(12).std() * sqrt(12) * 100</code>	Captures local risk conditions for RAG retrieval of historical analogues.
Normalised price index	Per-ticker: <code>price_idx = close / close.iloc[0] * 100</code>	Places tickers on comparable scale for cross-asset visualisations and retrieval.
Event-aligned windows	Construct return/volatility windows around sentiment timestamps	Creates historical episodes pairing text with price paths for FAISS-based RAG retrieval.
Exclusion from classifier	Price features used only for RAG and analysis, not DistilRoBERTa inputs	Keeps classifier focused on text while enabling price reasoning via retrieval layer.



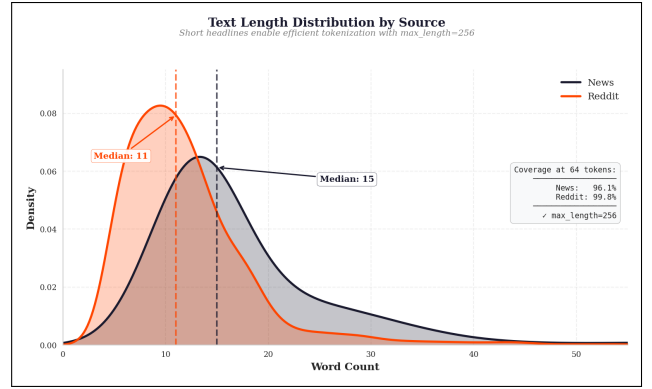
(a) Daily news sentiment with 7-day rolling average. Grey shows raw daily scores; red line shows smoothed trend. Event annotations highlight sentiment drivers: Trump/Musk DOGE news, Tesla Robotaxi hype, and NVDA Blackwell delays. Green/red fills mark periods exceeding ± 0.25 thresholds.



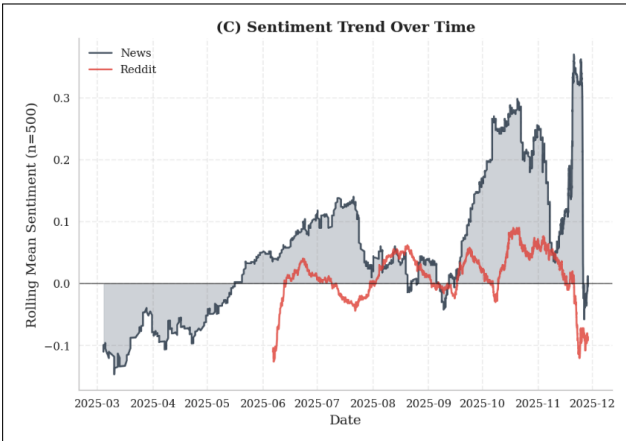
(b) Reddit sentiment by hour of day (UTC). Weekends (red) exhibit extreme volatility with a sharp positive spike at 7am UTC and deep negative drops at 4pm and 10pm, while weekdays (blue) show more stable patterns. Weekend panic selling validates the need for time-aware sentiment weighting.



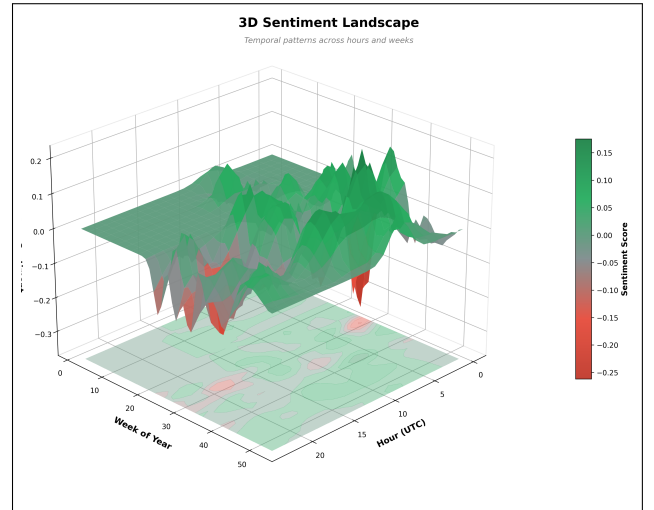
(c) Sentiment distribution by source. News exhibits stronger signals (32% positive, 20% negative) versus Reddit's neutral skew (74%), with significant difference ($t=10.86$, $p<0.001$) justifying weighted aggregation ($\alpha=0.6$, $\beta=0.4$).



(d) Text length distribution by source. Reddit posts are shorter (median 11 words) than news headlines (median 15). With 96–99% coverage at 64 tokens, `max_length=256` provides ample headroom.



(e) Rolling mean sentiment ($n=500$) over time. News sentiment (dark) trends increasingly positive toward November 2025, while Reddit (red) remains near-neutral with higher volatility.



(f) 3D sentiment landscape showing temporal patterns across hours (UTC) and weeks. Green peaks indicate positive sentiment clusters; red valleys show negative periods. Gaussian smoothing ($\sigma=1.2$) reveals underlying patterns.

Figure 4: Exploratory data analysis: (a) daily news sentiment trends with event annotations, (b) Reddit hourly sentiment showing weekend volatility and panic selling patterns, (c) sentiment distribution comparing news vs Reddit sources, (d) text length characteristics by source, (e) temporal sentiment trends over collection period, (f) 3D sentiment landscape across time dimensions.

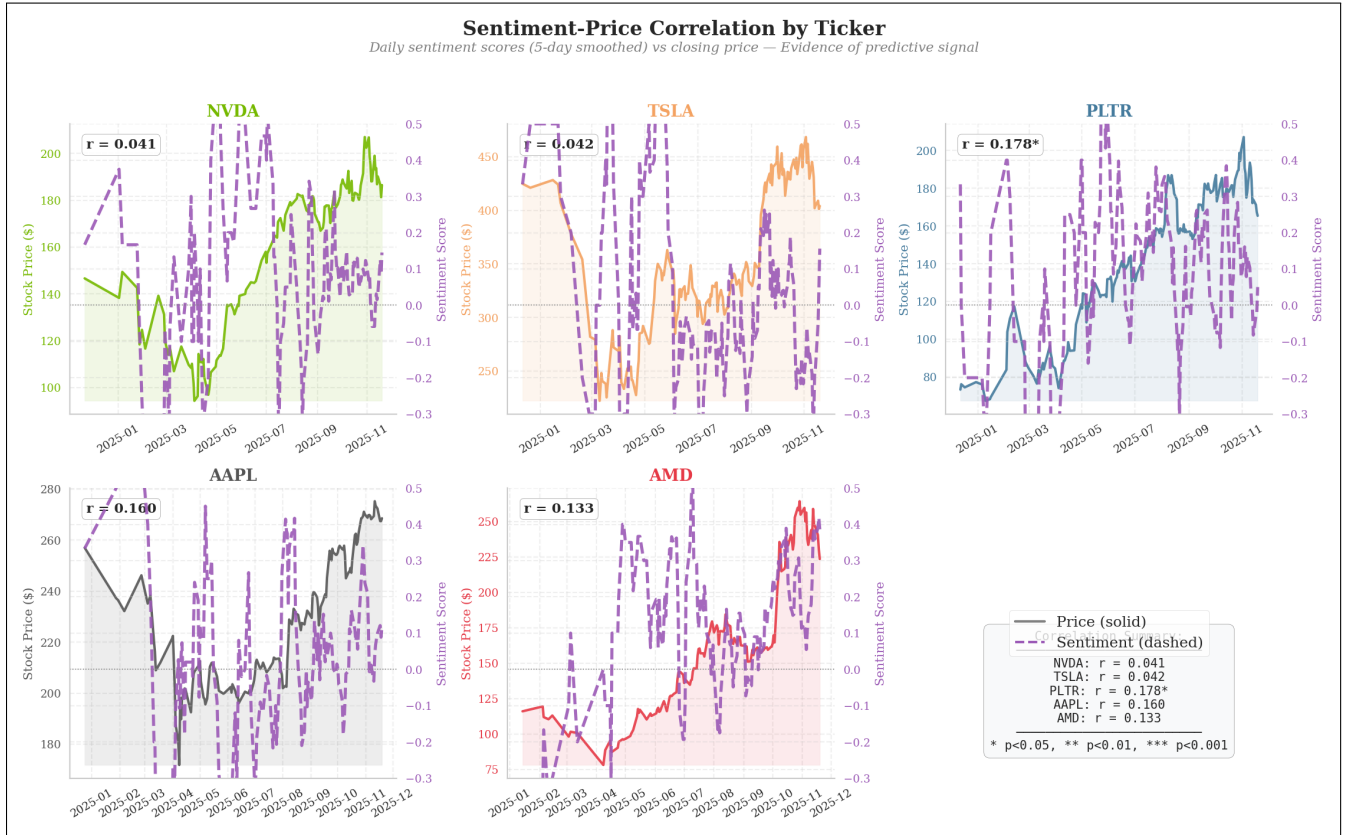


Figure 5: Sentiment-price correlation across five tickers. Solid lines: closing prices; dashed purple: 5-day smoothed sentiment. PLTR shows significant correlation ($r=0.178$, $p<0.05$). Visual co-movement validates sentiment’s predictive potential for the RAG retrieval system.

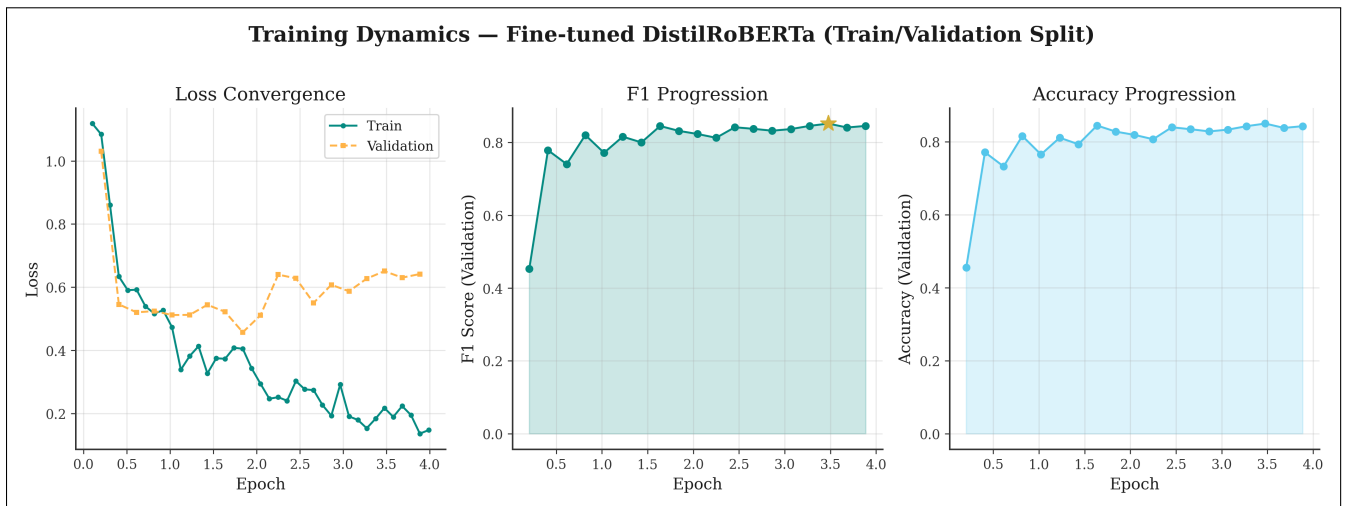


Figure 6: Training dynamics over 4 epochs: (a) train/validation loss convergence showing decreasing training loss with stable validation loss, (b) F1 score progression with best validation point marked by star (0.851), (c) accuracy stabilisation reaching 82.6% on test set. Rapid learning occurs in epoch 1 with minimal overfitting observed thereafter.

3.4 Exploratory Data Analysis

Exploratory analysis examined data quality, source characteristics, and sentiment-price relationships to validate preprocessing decisions and the weighted aggregation design (Figure 4).

3.5 Label Engineering

We transform noisy heterogeneous labels into a consistent three-class scheme and model IDs using rule-based mappings and strict filtering. (Table 6).

3.6 Feature Engineering

Feature engineering focuses on constructing robust text and market representations. We standardise the text field, apply transformer tokenisation, and derive complementary return and volatility features from 5-minute price bars, which support downstream RAG-based retrieval and diagnostic analysis (Table 7).

4. Model Training and Evaluation

4.1 Models

Table 8 summarises the six approaches evaluated, progressing from naive baselines through classical machine learning to transformer architectures.

4.2 Training Process

We fine-tune DistilRoBERTa-base (82M parameters) for three-class sentiment classification. The model processes tokenised headlines with maximum sequence length of 128 tokens. Training employs AdamW optimiser with learning rate 2×10^{-5} , batch size 16, weight decay 0.01, and linear warmup over 10% of 1,956 training steps across 4 epochs.

To address class imbalance, we implement weighted cross-entropy loss with inverse frequency weights computed from the training distribution (Negative: 2.09, Neutral: 0.53, Positive: 1.54). This penalises misclassification of minority classes proportionally, eliminating the need for resampling techniques.

For statistical rigour, all stochastic models run three times with different random seeds (42, 123, 456). We report mean $\pm$ standard deviation and apply McNemar’s test for pairwise classifier comparison with Cohen’s d effect size measurement.

4.3 Results

Table 9 presents comprehensive evaluation metrics across all models. Figure 8a visualises model comparison with error bars, Figure 8b shows the confusion matrix for the best model, and Figure 6 displays training dynamics.

The fine-tuned model achieves 83.1% F1, representing 132% improvement over random baseline. All pairwise comparisons show statistical significance ($p < 0.0001$) with large effect sizes ($d > 23$). Per-class performance remains balanced with 81–84% recall across sentiment categories (Figure 8b).

4.4 Justification of Results

The substantial performance gap between fine-tuned and zero-shot DistilRoBERTa (83.1% vs 21.1% F1) aligns with transfer learning theory, where domain-specific fine-tuning adapts pre-trained representations to target distributions (Howard & Ruder, 2018). Financial text exhibits distinct vocabulary and sentiment expressions. Phrases like “beat expectations” or “guidance cut” carry domain-specific polarity absent from general corpora (Malo et al., 2014).

The transformer’s superiority over TF-IDF (+8.6% F1) reflects contextual embeddings capturing word sense disambiguation, whereas bag-of-words approaches treat “Apple shares rise” and “apple prices rise” identically (Devlin et al., 2019). VADER’s underperformance (56.9% F1) confirms that lexicon-based methods struggle with implicit financial sentiment lacking explicit valence words (Loughran & McDonald, 2011).

5. Additional System Components

5.1 Weighted Aggregation

Daily sentiment scores undergo source-weighted aggregation where news receives higher weight ($\alpha=0.6$) than social media ($\beta=0.4$), reflecting the greater reliability of professional journalism over user-generated content.

5.2 RAG Implementation

The RAG component uses sentence-transformers to encode aggregated daily sentiment into 384-dimensional vectors, stored in a FAISS index with 54,642 historical price-sentiment pairs. At inference, cosine similarity retrieves the top-5 most similar historical periods. A similarity-weighted average of subsequent price movements generates directional predictions with confidence scores.

Encoding: $\mathbf{v}_t = \text{Encoder}(s_t) \in \mathbb{R}^{384}$

Similarity: $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}, \quad k = 5$

Prediction: $\hat{p}_{t+1} = \frac{\sum_{i \in \mathcal{N}_k} w_i \cdot \Delta p_i}{\sum_{i \in \mathcal{N}_k} w_i}$

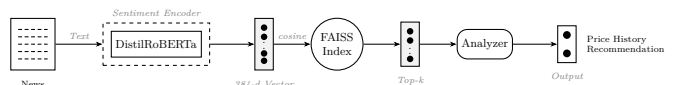


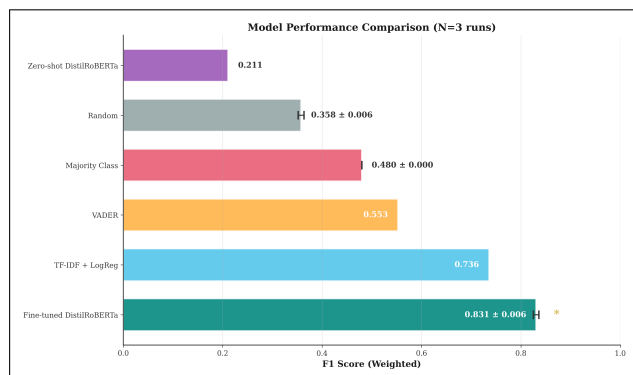
Figure 7: RAG retrieval pipeline. Sentiment vectors query FAISS index to retrieve historically similar periods. An analyzer produces price history and recommendations.

Table 8: Model descriptions. Approaches range from statistical baselines to domain-adapted transformers.

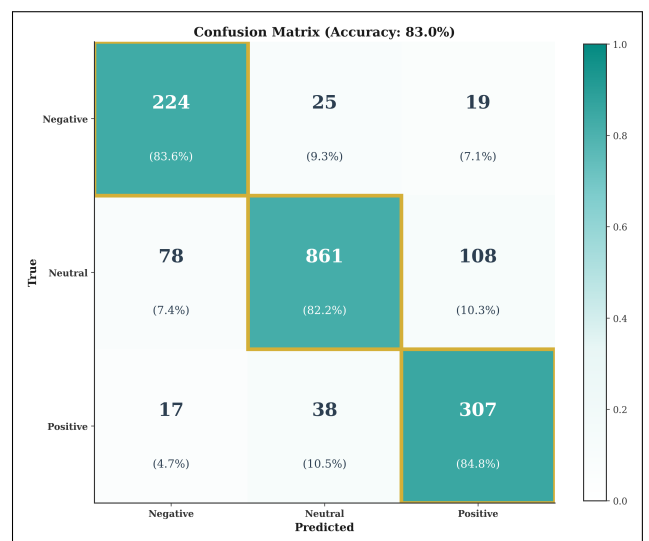
Model	Type	Description
Random	Baseline	Uniform random prediction across three classes
Majority Class	Baseline	Always predicts most frequent class (Neutral, 62.4%)
VADER	Lexicon	Rule-based sentiment analysis using valence-aware dictionary designed for social media text (Hutto & Gilbert, 2014)
TF-IDF + LogReg	Classical ML	Bigram TF-IDF features (5000 max) with L2-regularised logistic regression and balanced class weights (Pedregosa et al., 2011)
Zero-shot DistilRoBERTa	Transformer	Pre-trained NLI cross-encoder applied to sentiment without domain-specific fine-tuning (Reimers & Gurevych, 2019)
Fine-tuned RoBERTa	Distil-Transformer	Knowledge-distilled RoBERTa (82M parameters) fine-tuned on our financial sentiment corpus (Sanh et al., 2019)

Table 9: Comprehensive model evaluation (N=3 runs). All comparisons against Fine-tuned DistilRoBERTa show statistical significance (McNemar’s test, $p < 0.0001$ for all) with large effect sizes. Best results in **bold**. Std column shows standard deviation range across metrics.

Model	N	Accuracy	F1 (W)	F1 (M)	Prec (W)	Rec (W)	Std	Cohen’s d
Fine-tuned DistilRoBERTa	3	0.826	0.831	0.804	0.838	0.826	0.006	—
TF-IDF + LogReg	3	0.731	0.736	0.683	0.743	0.731	0.000	23.7
VADER	3	0.537	0.553	0.495	0.576	0.537	0.000	69.7
Majority Class	3	0.624	0.480	0.256	0.390	0.624	0.000	87.9
Random	3	0.329	0.358	0.302	0.445	0.329	0.006–0.008	82.4
Zero-shot DistilRoBERTa	3	0.321	0.211	0.315	0.652	0.321	0.000	155.4



(a) Model performance comparison with F1 scores and error bars (N=3 runs). Fine-tuned DistilRoBERTa achieves 0.831 ± 0.006 , significantly outperforming all baselines ($p < 0.0001$). Star indicates best model.



(b) Confusion matrix for fine-tuned DistilRoBERTa (accuracy: 82.6%). Gold borders highlight correct predictions. Per-class recall: Negative 83.1%, Neutral 81.3%, Positive 84.2%.

6. Deployment and Theme Extraction

6.1 Cloud Deployment

StockFeelings is deployed as a cloud-native application using Google Cloud Platform services. The architecture separates model storage from compute, enabling independent scaling and updates. The fine-tuned DistilRoBERTa model is packaged and uploaded to Google Cloud Storage (Standard class, multi-region) as a publicly accessible artifact. During container startup, the Docker image retrieves this model for inference.

The backend API (FastAPI/Python) and frontend (Vite/React) are containerised and deployed to Google Cloud Run with continuous deployment from GitHub. Table 13 and Table 14 summarise the deployment configuration. The serverless architecture scales from zero to 100 instances based on demand, with cold starts mitigated by extended startup probes accommodating model loading time. Figure 10 shows the deployed application interface.

6.2 Theme Extraction and FAISS Indexing

Financial text contains thematic signals beyond sentiment polarity. We extract 12 research-backed theme categories derived from the Loughran-McDonald Master Dictionary (Loughran & McDonald, 2011) and SEC Form 8-K material event classifications (U.S. Securities and Exchange Commission, 2024). Theme extraction employs a hybrid approach combining keyword matching (60% weight) with BART-large-mnli zero-shot classification (40% weight) (M. Lewis et al., 2019). Keyword matching provides deterministic, interpretable scores using domain-specific dictionaries, while BART captures contextual semantics where explicit keywords are absent.

Text data is aggregated into 4-hour windows aligned with market sessions (Table 11). For each window, we compute theme scores normalised to $[0, 1]$ and extract salient keywords using KeyBERT (Grootendorst, 2020). The resulting 13-dimensional vectors (12 themes + sentiment) are L2-normalised and indexed using Facebook AI Similarity Search (FAISS) with inner product search, enabling millisecond-scale cosine similarity retrieval across 3,316 historical periods.

Table 10: Research-backed theme categories for financial text classification.

Tier	Theme	Source
Core Financial	Earnings	L-M Dictionary
	Analyst	Market research
	Valuation	Baker & Wurgler
Corporate Events	M&A	SEC 8-K 2.01
	Executive	SEC 8-K 5.02
	Legal/Reg.	L-M Litigious
Product & Tech	Product Launch	Domain-specific
	AI/Tech Supply Chain	Domain-specific
Market & Macro	Price Action	Domain-specific
	Competition Macro	Domain-specific Tetlock

Table 11: 4-hour window alignment with market sessions (EST).

Window	Hours	Market Context
0	00:00–04:00	Overnight
4	04:00–08:00	Pre-market
8	08:00–12:00	Morning session
12	12:00–16:00	Afternoon (close)
16	16:00–20:00	After-hours
20	20:00–24:00	Evening

Table 12: Hybrid theme extraction pipeline.

Component	Weight	Description
Keyword Matching	0.6	Domain dictionary with 200+ terms per theme
BART Zero-shot	0.4	facebook/bart-large-mnli (406M params)
KeyBERT	–	Top-5 keyword extraction (all-MiniLM-L6-v2)
<i>Final score: $S_{\text{theme}} = 0.6 \cdot S_{\text{kw}} + 0.4 \cdot S_{\text{BART}}$</i>		

Table 13: Model storage configuration

Property	Value
Service	Google Cloud Storage
Storage Class	Standard
Location	Multi-region
Access	Public
Model URL	Click to download model

Table 14: Cloud Run deployment configuration

Parameter	Value
Memory	512 MB
CPU	1 vCPU
Request Timeout	300s
Startup Probe	TCP 8000 / 240s
Concurrent Requests	80 per instance
Min Instances	0 (scale to zero)
Max Instances	100
Live App URL	Open Live App
Backend URL	API Endpoint
<i>Note: Click “sync” on dashboard to fetch new data</i>	

7. Conclusion

StockFeelings demonstrates that domain-specific fine-tuning substantially outperforms general-purpose sentiment approaches, achieving 83.1% F1 score, a 132% improvement over random baseline. The significant difference between news and Reddit sentiment distributions ($t=10.86$, $p<0.001$) empirically validates source-weighted aggregation. The core innovation, using RAG to answer “when sentiment looked like this before, what happened next?”, bridges real-time analysis with historical context.

Limitations include coverage of only five technology stocks and a 12-month collection period. Future work will expand ticker coverage, incorporate SEC filings, and evaluate prediction accuracy against actual price movements. Figure 9 shows the deployed application interface across different sentiment states.

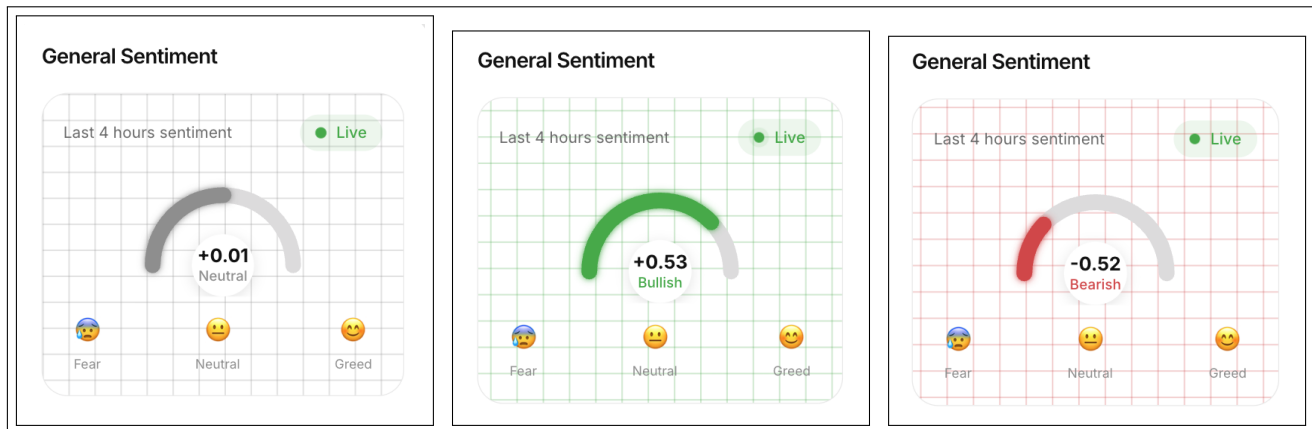


Figure 9: StockFeelings web interface showing sentiment dashboards for neutral (left), positive (center), and negative (right) sentiment predictions with historical context matches and news highlights. Screenshots display synthetic data for demonstration purposes.

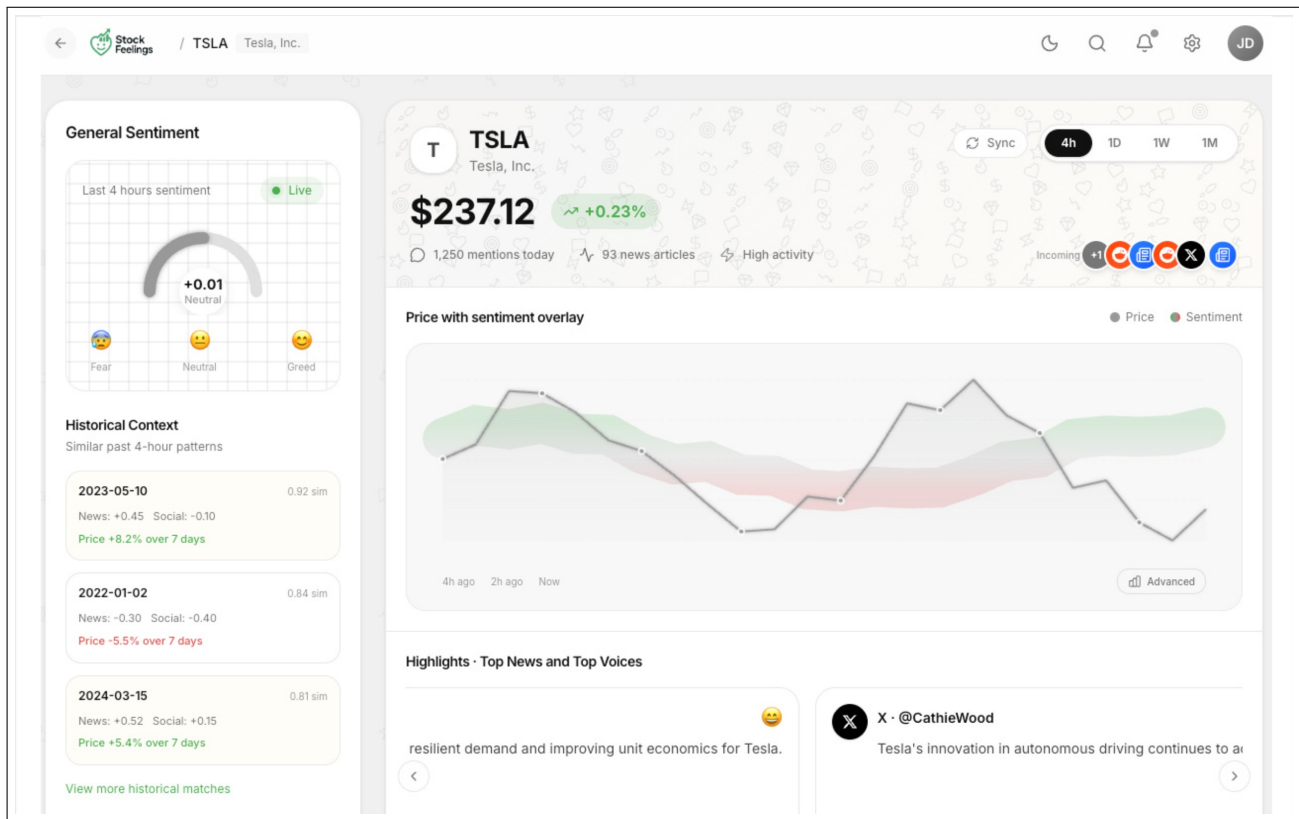


Figure 10: Complete StockFeelings dashboard showing TSLA sentiment analysis with price overlay, historical context matches, and news highlights. Screenshot displays synthetic data for demonstration purposes.

References

- Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1), 1–8. <https://doi.org/10.1016/j.jocs.2010.12.007>
- Buffett, W. (2008). Buy american. i am. *The New York Times*. <https://www.nytimes.com/2008/10/17/opinion/17buffett.html>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*. <https://arxiv.org/abs/1810.04805>
- Grootendorst, M. (2020). Keybert: Minimal keyword extraction with bert.
- Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 328–339.
- Hutto, C., & Gilbert, E. (2014). Vader: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*, 8(1), 216–225.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2019). Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Loughran, T., & McDonald, B. (2011). When is a liability not a liability? textual analysis, dictionaries, and 10-ks. *The Journal of Finance*, 66(1), 35–65. <https://doi.org/10.1111/j.1540-6261.2010.01625.x>
- Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4), 782–796.
- Pagolu, V. S., Reddy, K. N., Panda, G., & Majhi, B. (2016). Sentiment analysis of twitter data for predicting stock market movements. *2016 International Conference on Signal Processing, Communication, Power and Embedded System (SCOPEs)*, 1345–1350. <https://doi.org/10.1109/SCOPEs.2016.7955659>
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. *Proceedings of EMNLP-IJCNLP*, 3982–3992.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*. <https://arxiv.org/abs/1910.01108>
- Sigalos, M. (2025). *Openai unveils chatgpt atlas browser, sending alphabet shares lower*. CNBC. Retrieved October 21, 2025, from <https://www.cnbc.com/2025/10/21/openai-browser-alphabet-stock.html>
- U.S. Securities and Exchange Commission. (2024). Form 8-k.